VIDEO E TATTICA;

```
import React, { useEffect, useRef, useState } from 'react';
import { base44 } from '@/api/base44Client';
import { useRole } from '@/context/RoleContext';
import { SectionTitle, Badge, EmptyState } from '@/components/ui-shared';
import { Video, PenTool, Plus, Trash2, Save, Upload, ArrowRight, Circle, Users, Eraser }
from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Textarea } from '@/components/ui/textarea';

const canUpload = ['presidente', 'allenatore1', 'allenatore2'];
const canSaveSchema = ['presidente', 'allenatore1', 'allenatore2'];

const INITIAL_PLAYERS = [
 { id: 4, x: 50, y: 45 },
 { id: 3, x: 28, y: 45 },
 { id: 2, x: 72, y: 45 },
 { id: 5, x: 28, y: 55 },
 { id: 6, x: 50, y: 55 },
 { id: 1, x: 72, y: 55 },
];

function youtubeId(url) {
 const m = url.match(/(?:youtube\.com\/(?:watch\?v=|embed\/)|youtu\.be\/)([\w-]{11})/);
 return m ? m[1] : null;
}

export default function VideoTattica() {
 const { role } = useRole();
 const [tab, setTab] = useState('video');

 // ---- VIDEO ----
 const [clips, setClips] = useState([]);
 const [loadingV, setLoadingV] = useState(true);
 const [selClip, setSelClip] = useState(null);
 const [showForm, setShowForm] = useState(false);
 const [form, setForm] = useState({ titolo: '', descrizione: '', url_video: '', tipo:
'Partita', categoria: '' });

 const loadClips = () => {
  base44.entities.VideoClip.list('-data').then((r) => { setClips(r);
setLoadingV(false); }).catch(() => setLoadingV(false));
 };
 useEffect(() => { loadClips(); }, []);

 const addClip = async () => {
  if (!form.titolo || !form.url_video) return;
  await base44.entities.VideoClip.create({ ...form, caricato_da: role.nome, data: new
Date().toISOString().slice(0, 10) });
  setForm({ titolo: '', descrizione: '', url_video: '', tipo: 'Partita', categoria:
'' });
```

```
  setShowForm(false);
  loadClips();
 };

 // ---- LAVAGNA ----
 const svgRef = useRef(null);
 const [players, setPlayers] = useState(INITIAL_PLAYERS);
 const [arrows, setArrows] = useState([]);
 const [drawMode, setDrawMode] = useState(false);
 const [arrowColor, setArrowColor] = useState('#16a34a');
 const [drawStart, setDrawStart] = useState(null);
 const [dragging, setDragging] = useState(null);
 const [note, setNote] = useState('');
 const [schemaNome, setSchemaNome] = useState('');
 const [schemaTipo, setSchemaTipo] = useState('Ricezione');
 const [schemi, setSchemi] = useState([]);
 const [loadingS, setLoadingS] = useState(true);

 const loadSchemi = () => {
  base44.entities.SchemaTattico.list('-data').then((r) => { setSchemi(r);
setLoadingS(false); }).catch(() => setLoadingS(false));
 };
 useEffect(() => { loadSchemi(); }, []);

 const toSvg = (e) => {
  const rect = svgRef.current.getBoundingClientRect();
  return {
   x: ((e.clientX - rect.left) / rect.width) * 100,
   y: ((e.clientY - rect.top) / rect.height) * 60,
  };
 };

 const onPlayerDown = (e, id) => {
  if (drawMode) return;
  e.stopPropagation();
  setDragging(id);
 };

 const onSvgMove = (e) => {
  if (dragging == null) return;
  const p = toSvg(e);
  setPlayers((prev) => prev.map((pl) => pl.id === dragging ? { ...pl, x: Math.max(4,
Math.min(96, p.x)), y: Math.max(32, Math.min(58, p.y)) } : pl));
 };

 const onSvgUp = () => setDragging(null);

 const onSvgClick = (e) => {
  if (!drawMode) return;
  const p = toSvg(e);
  if (!drawStart) {
   setDrawStart(p);
```

```
  } else {
   setArrows((prev) => [...prev, { x1: drawStart.x, y1: drawStart.y, x2: p.x, y2: p.y,
color: arrowColor }]);
   setDrawStart(null);
  }
 };

 const clearAll = () => { setArrows([]); setDrawStart(null); };
 const resetPlayers = () => setPlayers(INITIAL_PLAYERS);

 const saveSchema = async () => {
  if (!schemaNome) return;
  await base44.entities.SchemaTattico.create({
   nome: schemaNome, tipo: schemaTipo, note,
   config: JSON.stringify({ players, arrows }),
   creato_da: role.nome, data: new Date().toISOString().slice(0, 10),
  });
  setSchemaNome('');
  loadSchemi();
 };

 const loadSchema = (s) => {
  try {
   const c = JSON.parse(s.config);
   if (c.players) setPlayers(c.players);
   if (c.arrows) setArrows(c.arrows);
   setNote(s.note || '');
   setSchemaNome(s.nome);
   setSchemaTipo(s.tipo);
  } catch { /* ignore */ }
 };

 return (
  <div>
   <SectionTitle title="Video Analisi & Lavagna Tattica" subtitle="Clip delle partite e
schemi di ricezione, difesa e attacco." />

   <div className="flex gap-2 mb-6 bg-muted/50 p-1 rounded-xl w-fit">
    <button onClick={() => setTab('video')} className={`px-4 py-2 rounded-lg text-sm
font-semibold flex items-center gap-2 transition ${tab === 'video' ? 'bg-card shadow
text-foreground' : 'text-muted-foreground'}`}>
     <Video className="w-4 h-4" /> Video Analisi
    </button>
    <button onClick={() => setTab('lavagna')} className={`px-4 py-2 rounded-lg text-sm
font-semibold flex items-center gap-2 transition ${tab === 'lavagna' ? 'bg-card shadow
text-foreground' : 'text-muted-foreground'}`}>
     <PenTool className="w-4 h-4" /> Lavagna Tattica
    </button>
   </div>

   {tab === 'video' && (
    <div>
```

```
      {canUpload.includes(role.key) && (
       <div className="mb-4">
        {!showForm ? (
         <Button onClick={() => setShowForm(true)} className="bg-amber-500 text-
[hsl(222,47%,14%)] hover:bg-amber-600"><Plus className="w-4 h-4 mr-1" /> Aggiungi
clip</Button>
        ) : (
         <div className="bg-card rounded-2xl border p-5 space-y-3">
          <div className="grid sm:grid-cols-2 gap-3">
           <Input value={form.titolo} onChange={(e) => setForm({ ...form, titolo:
e.target.value })} placeholder="Titolo (es. Ricezione Set 3 vs Roma)" />
           <select value={form.tipo} onChange={(e) => setForm({ ...form, tipo:
e.target.value })} className="px-3 py-2 rounded-lg border bg-card text-sm">
            {['Partita', 'Avversario', 'Esercizio', 'Tattica'].map((t) => <option key={t}
value={t}>{t}</option>)}
           </select>
          </div>
          <Input value={form.url_video} onChange={(e) => setForm({ ...form, url_video:
e.target.value })} placeholder="URL YouTube o link video" />
          <Input value={form.categoria} onChange={(e) => setForm({ ...form, categoria:
e.target.value })} placeholder="Categoria (es. Battuta avversaria)" />
          <Textarea value={form.descrizione} onChange={(e) => setForm({ ...form,
descrizione: e.target.value })} placeholder="Note del coach..." rows={2} />
          <div className="flex gap-2">
           <Button onClick={addClip} className="bg-amber-500 text-[hsl(222,47%,14%)]
hover:bg-amber-600"><Upload className="w-4 h-4 mr-1" /> Carica</Button>
           <Button variant="outline" onClick={() => setShowForm(false)}>Annulla</Button>
          </div>
         </div>
        )}
       </div>
      )}

      {selClip && (
       <div className="fixed inset-0 z-50 bg-black/80 flex items-center justify-center p-
4" onClick={() => setSelClip(null)}>
        <div className="bg-card rounded-2xl w-full max-w-3xl overflow-hidden" onClick={(e)
=> e.stopPropagation()}>
         {youtubeId(selClip.url_video) ? (
          <div className="aspect-video bg-black">
           <iframe src={`https://www.youtube.com/embed/${youtubeId(selClip.url_video)}`}
className="w-full h-full" allowFullScreen title={selClip.titolo} />
          </div>
         ) : (
          <div className="aspect-video bg-black flex items-center justify-center">
           <video src={selClip.url_video} controls className="w-full h-full" />
          </div>
         )}
         <div className="p-5">
          <div className="flex items-center gap-2 mb-1">
           <Badge variant="info">{selClip.tipo}</Badge>
           <span className="text-xs text-muted-foreground">{selClip.caricato_da}</span>
          </div>
```

```
          <h3 className="font-bold text-lg">{selClip.titolo}</h3>
          {selClip.descrizione && <p className="text-sm text-muted-foreground mt-1
whitespace-pre-wrap">{selClip.descrizione}</p>}
          <Button onClick={() => setSelClip(null)} className="w-full mt-
4">Chiudi</Button>
         </div>
        </div>
       </div>
      )}

      {loadingV ? (
       <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-4">{[...Array(3)].map((_, i)
=> <div key={i} className="h-48 rounded-2xl bg-muted animate-pulse" />)}</div>
      ) : clips.length === 0 ? (
       <EmptyState icon={Video} title="Nessuna clip" subtitle="Lo staff tecnico può
caricare video di partite e avversari." />
      ) : (
       <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-4">
        {clips.map((c) => {
         const yt = youtubeId(c.url_video);
         return (
          <button key={c.id} onClick={() => setSelClip(c)} className="text-left bg-card
rounded-2xl border overflow-hidden hover:border-amber-400 hover:shadow-md transition">
           <div className="aspect-video bg-gradient-to-br from-[hsl(222,47%,14%)] to-
[hsl(222,47%,28%)] relative flex items-center justify-center">
            {yt ? (
             <img src={`https://img.youtube.com/vi/${yt}/hqdefault.jpg`} alt=""
className="w-full h-full object-cover" />
            ) : (
             <Video className="w-12 h-12 text-amber-400/50" />
            )}
            <div className="absolute inset-0 flex items-center justify-center bg-
black/20">
             <div className="w-12 h-12 rounded-full bg-amber-500 flex items-center
justify-center"><Video className="w-6 h-6 text-[hsl(222,47%,14%)]" /></div>
            </div>
           </div>
           <div className="p-4">
            <Badge variant="info">{c.tipo}</Badge>
            <h3 className="font-bold mt-2 line-clamp-1">{c.titolo}</h3>
            {c.descrizione && <p className="text-xs text-muted-foreground mt-1 line-
clamp-2">{c.descrizione}</p>}
           </div>
          </button>
         );
        })}
       </div>
      )}
     </div>
    )}

    {tab === 'lavagna' && (
     <div className="grid lg:grid-cols-3 gap-6">
```

```
      <div className="lg:col-span-2 bg-card rounded-2xl border p-4">
       <div className="flex items-center gap-2 mb-3 flex-wrap">
        <Button size="sm" variant={drawMode ? 'default' : 'outline'} onClick={() =>
{ setDrawMode((d) => !d); setDrawStart(null); }} className={drawMode ? 'bg-amber-500
text-[hsl(222,47%,14%)]' : ''}>
         <PenTool className="w-4 h-4 mr-1" /> {drawMode ? 'Disegno ON' : 'Disegna
frecce'}
        </Button>
        {drawMode && (
         <div className="flex items-center gap-1">
          {['#16a34a', '#dc2626', '#2563eb', '#f59e0b'].map((c) => (
           <button key={c} onClick={() => setArrowColor(c)} className={`w-6 h-6 rounded-
full border-2 ${arrowColor === c ? 'ring-2 ring-offset-1 ring-slate-400' : 'border-
white'}`} style={{ background: c }} />
          ))}
         </div>
        )}
        <Button size="sm" variant="outline" onClick={clearAll}><Eraser className="w-4 h-4
mr-1" /> Pulisci frecce</Button>
        <Button size="sm" variant="outline" onClick={resetPlayers}><Users className="w-4
h-4 mr-1" /> Reset giocatori</Button>
        {drawStart && <span className="text-xs text-amber-600 font-semibold">Clicca il
punto finale della freccia...</span>}
       </div>

       <div className="relative w-full" style={{ aspectRatio: '100/60' }}>
        <svg
         ref={svgRef}
         viewBox="0 0 100 60"
         className="w-full h-full rounded-xl bg-gradient-to-b from-amber-50 to-amber-100
border-4 border-[hsl(222,47%,18%)] touch-none"
         onMouseMove={onSvgMove}
         onMouseUp={onSvgUp}
         onMouseLeave={onSvgUp}
         onClick={onSvgClick}
         style={{ cursor: drawMode ? 'crosshair' : 'default' }}
        >
         {/* campo */}
         <line x1="0" y1="30" x2="100" y2="30" stroke="#1e293b" strokeWidth="0.6" />
         <line x1="30" y1="30" x2="30" y2="60" stroke="#1e293b" strokeWidth="0.4" />
         <line x1="70" y1="30" x2="70" y2="60" stroke="#1e293b" strokeWidth="0.4" />
         <line x1="50" y1="30" x2="50" y2="60" stroke="#1e293b" strokeWidth="0.4" />
         <line x1="0" y1="60" x2="100" y2="60" stroke="#1e293b" strokeWidth="0.6" />
         <line x1="0" y1="0" x2="100" y2="0" stroke="#1e293b" strokeWidth="0.6" />
         <line x1="0" y1="0" x2="0" y2="60" stroke="#1e293b" strokeWidth="0.6" />
         <line x1="100" y1="0" x2="100" y2="60" stroke="#1e293b" strokeWidth="0.6" />
         <text x="50" y="29" textAnchor="middle" fontSize="2.5" fill="#1e293b"
opacity="0.4">RETE</text>
         {/* frecce */}
         {arrows.map((a, i) => (
          <g key={i}>
           <line x1={a.x1} y1={a.y1} x2={a.x2} y2={a.y2} stroke={a.color}
strokeWidth="0.8" markerEnd="url(#arrowhead)" />
```

```
          <circle cx={a.x1} cy={a.y1} r="0.8" fill={a.color} />
         </g>
        ))}
        {drawStart && <circle cx={drawStart.x} cy={drawStart.y} r="0.8"
fill={arrowColor} opacity="0.5" />}
        <defs>
         <marker id="arrowhead" markerWidth="4" markerHeight="4" refX="2" refY="1.5"
orient="auto">
          <polygon points="0 0, 3 1.5, 0 3" fill="context-stroke" />
         </marker>
        </defs>
        {/* giocatori */}
        {players.map((p) => (
         <g
          key={p.id}
          onMouseDown={(e) => onPlayerDown(e, p.id)}
          style={{ cursor: drawMode ? 'default' : 'grab' }}
         >
          <circle cx={p.x} cy={p.y} r="3.5" fill="#1e293b" stroke="#f59e0b"
strokeWidth="0.8" />
          <text x={p.x} y={p.y + 1.2} textAnchor="middle" fontSize="3" fontWeight="bold"
fill="white">{p.id}</text>
         </g>
        ))}
       </svg>
      </div>
      <p className="text-xs text-muted-foreground mt-2">
       {drawMode ? 'Modalità disegno: clicca due punti per creare una freccia (movimento
o traiettoria palla).' : 'Trascina i giocatori per posizionarli. Attiva "Disegna frecce"
per tracciare schemi.'}
      </p>
     </div>

     <div className="space-y-4">
      <div className="bg-card rounded-2xl border p-5">
       <h3 className="font-bold mb-3 flex items-center gap-2"><Save className="w-4 h-4
text-amber-500" /> Salva schema</h3>
       {canSaveSchema.includes(role.key) ? (
        <div className="space-y-3">
         <Input value={schemaNome} onChange={(e) => setSchemaNome(e.target.value)}
placeholder="Nome schema" />
         <select value={schemaTipo} onChange={(e) => setSchemaTipo(e.target.value)}
className="w-full px-3 py-2 rounded-lg border bg-card text-sm">
          {['Ricezione', 'Difesa', 'Attacco', 'Battuta', 'Libero'].map((t) => <option
key={t}>{t}</option>)}
         </select>
         <Textarea value={note} onChange={(e) => setNote(e.target.value)}
placeholder="Note tattiche..." rows={3} />
         <Button onClick={saveSchema} className="w-full bg-amber-500 text-
[hsl(222,47%,14%)] hover:bg-amber-600"><Save className="w-4 h-4 mr-1" /> Salva</Button>
        </div>
       ) : (
        <p className="text-sm text-muted-foreground">Solo lo staff tecnico può salvare
```

```
schemi. Puoi visualizzare quelli esistenti.</p>
        )}
       </div>

       <div className="bg-card rounded-2xl border p-5">
        <h3 className="font-bold mb-3 flex items-center gap-2"><PenTool className="w-4 h-4
text-amber-500" /> Schemi salvati</h3>
        {loadingS ? (
         <div className="space-y-2">{[...Array(2)].map((_, i) => <div key={i}
className="h-12 rounded-lg bg-muted animate-pulse" />)}</div>
        ) : schemi.length === 0 ? (
         <p className="text-sm text-muted-foreground">Nessuno schema salvato.</p>
        ) : (
         <div className="space-y-2">
          {schemi.map((s) => (
           <button key={s.id} onClick={() => loadSchema(s)} className="w-full text-left
p-3 rounded-lg bg-muted/40 hover:bg-amber-50 border hover:border-amber-300 transition">
            <div className="flex items-center justify-between">
             <span className="font-semibold text-sm">{s.nome}</span>
             <Badge variant="info">{s.tipo}</Badge>
            </div>
            {s.note && <p className="text-xs text-muted-foreground mt-1 line-clamp-
1">{s.note}</p>}
            <p className="text-xs text-muted-foreground mt-1">{s.creato_da} · {new
Date(s.data).toLocaleDateString('it-IT')}</p>
           </button>
          ))}
         </div>
        )}
       </div>
      </div>
     </div>
    )}
   </div>
  );
}
```